🏠 Advanced Real Estate Valuation — Scale-Up Plan

> **Course**: Big Data Analytics (BDA) — Sem 6 **Current State**: King County dataset (~21,600 rows), Pandas + Scikit-Learn + XGBoost, single notebook **Target State**: Multi-million row dataset, spatial database, scalable ML, interactive dashboard

Phase Overview

Phase	Name	Estimated Effort	Key Deliverable
1	Data Acquisition & Storage Layer	1–2 days	Cleaned, partitioned Parquet dataset + DuckDB analytics
2	Advanced Feature Engineering	1–2 days	H3 spatial features, macro indicators, amenity proximity
3	Scalable ML Pipeline	1–2 days	LightGBM + Quantile models trained on millions of rows
4	Explainability & Deep Analysis	1 day	SHAP explanations, anomaly-filtered arbitrage detector
5	Interactive Dashboard & API	1–2 days	Streamlit app with maps, risk bands, and live valuation

Phase 1 — Data Acquisition & Storage Layer

> **IMPORTANT**
>
> This phase replaces the single 21K-row CSV with a production-scale dataset and an analytical storage engine capable of querying millions of rows in milliseconds.

1.1 Dataset Selection & Download

Option	Rows	Size	Source	Best For
USA Real Estate (Realtor.com)	**~2.2M**	~600 MB	Kaggle	Multi-state, directly compatible schema
UK Land Registry Price Paid	~29M	~4.2 GB	GOV.UK Open Data	Massive scale, multi-decade temporal
NYC PLUTO + Property Sales	~1M parcels	~1.5 GB	NYC OpenData	Rich geospatial polygons & zoning

Recommendation: Use the **USA Real Estate Dataset** — it is the largest single-country residential dataset with beds, baths, sqft, price, city, state, zip, and coordinates. It scales your project by **~100x** while keeping the feature schema familiar.

1.2 Raw Data Ingestion Script

Create `scripts/01_ingest.py`:

```
python
# Download from Kaggle → load into Pandas (chunked) → basic cleaning
# Steps:
#   1. Read CSV in chunks of 500K rows (avoids OOM on large files)
#   2. Drop rows with null price or null sqft
#   3. Remove extreme outliers (price < $10K or > $50M)
#   4. Standardize column names to snake_case
#   5. Parse and normalize date columns
#   6. Add sale_year, sale_month columns
```

1.3 Convert to Partitioned Parquet

Create `scripts/02_to_parquet.py`:

```
data/
├── properties/
│   ├── state=CA/
│   │   ├── sale_year=2023/
│   │   │   └── part-00001.parquet
│   │   └── sale_year=2024/
│   │       └── part-00001.parquet
│   ├── state=NY/
│   │   └── ...
│   └── state=WA/
│       └── ...
```

Why Parquet?

- **Columnar compression** — the ~600 MB CSV becomes ~150–200 MB Parquet (snappy).
- **Predicate pushdown** — querying `WHERE state = 'WA'` reads only that partition folder, skipping millions of irrelevant rows.
- **Column pruning** — selecting 5 columns out of 20 reads only those 5 from disk.

1.4 Set Up DuckDB as the Analytical Engine

Create `scripts/03_duckdb_setup.py`:

```
python
import duckdb

con = duckdb.connect("data/realestate.duckdb")

# Register the Parquet dataset as a virtual table
con.execute("""
    CREATE OR REPLACE VIEW properties AS
    SELECT * FROM read_parquet('data/properties/**/*.parquet', hive_partitioning=true)
""")

# Verify row count & schema
con.execute("SELECT count(*) FROM properties").fetchone()
# → (2,226,382,)

# Example: Median price by state
con.execute("""
    SELECT state, median(price) AS median_price, count(*) AS n_sales
    FROM properties
    WHERE price BETWEEN 10000 AND 10000000
    GROUP BY state
    ORDER BY median_price DESC
    LIMIT 10
""").df()
```

Why DuckDB?

- Zero setup — it is an embedded database (single pip install, no server).
- Processes multi-GB Parquet files in under 1 second.
- Returns results directly as Pandas DataFrames.
- Perfect for a BDA course — demonstrates **OLAP query processing** without Hadoop/Spark infrastructure.

1.5 Phase 1 Deliverables

- Raw dataset downloaded and stored locally
- Cleaning script with chunk-based processing
- Partitioned Parquet files (by state + year)
- DuckDB database with registered views
- Verification notebook: row counts, schema, sample queries, data quality stats

Phase 2 — Advanced Feature Engineering at Scale

> NOTE
>
> This phase upgrades your existing 3 novelties and adds new spatial/temporal features. All feature engineering runs through DuckDB SQL or vectorized operations on Parquet — no row-by-row Python loops.

2.1 Uber H3 Hexagonal Spatial Indexing (Replaces Geohash)

Current: `pygeohash.encode(lat, lon, precision=6)` → rectangular cells with uneven neighbor distances.

Upgrade: Uber H3 — a discrete global hexagonal grid.

Resolution	Avg Hex Area	Use Case in This Project
5	~252 km²	State / metro region aggregates
7	~5.16 km²	Neighborhood / suburb clusters
9	~0.105 km²	Street-level micro-markets

```
python
import h3

# Vectorized H3 indexing
df['h3_res7'] = df.apply(lambda r: h3.latlng_to_cell(r['lat'], r['lng'], 7), axis=1)
df['h3_res9'] = df.apply(lambda r: h3.latlng_to_cell(r['lat'], r['lng'], 9), axis=1)

# Compute micro-market features per hexagon
h3_agg = df.groupby('h3_res9').agg(
    micro_median_price=('price', 'median'),
    micro_median_ppsf=('price_per_sqft', 'median'),  # price per sqft
    micro_sales_volume=('price', 'count'),
    micro_avg_grade=('grade', 'mean')
).reset_index()

df = df.merge(h3_agg, on='h3_res9', how='left')
```

Why H3 over Geohash?

- All 6 neighbors are equidistant (isotropic spatial analysis).

- Hierarchical: each parent hex contains exactly 7 children — enables multi-resolution aggregation.
- Industry standard used by Uber, Carto, and Zillow.

2.2 Proximity & Amenity Features via OpenStreetMap

Using the `osmnx` library or the Overpass API, extract:

Feature	Description	Expected Impact on Price
`dist_to_nearest_school_km`	Haversine distance to closest school	Strong positive (good school district premium)
`n_schools_within_2km`	Count of schools in 2 km radius	Moderate positive
`dist_to_nearest_transit_km`	Distance to subway/metro/bus station	Negative correlation (closer = higher price)
`dist_to_downtown_km`	Distance to city center	Negative correlation
`n_parks_within_1km`	Count of parks nearby	Moderate positive

```
python
# Pre-compute a spatial index of POIs per city (one-time)
# Then do a vectorized nearest-neighbor join using BallTree
from sklearn.neighbors import BallTree
import numpy as np

# Build tree from school coordinates
school_tree = BallTree(np.radians(school_coords), metric='haversine')
distances, _ = school_tree.query(np.radians(property_coords), k=1)
df['dist_to_nearest_school_km'] = distances[:, 0] * 6371  # Earth radius
```

2.3 Temporal & Macroeconomic Indicators

Pull public economic time-series data to capture market-wide trends:

Indicator	Source	Granularity
30-Year Mortgage Rate	FRED API (`MORTGAGE30US`)	Weekly
Consumer Price Index	FRED API (`CPIAUCSL`)	Monthly
Unemployment Rate	FRED API (`UNRATE`)	Monthly
Housing Starts	FRED API (`HOUST`)	Monthly

```
python
from fredapi import Fred
fred = Fred(api_key='YOUR_KEY')

mortgage_rate = fred.get_series('MORTGAGE30US')
# Merge onto property data by sale_month
```

Impact: These features let the model learn that prices rise when mortgage rates drop (demand surge) and flatten during high-unemployment periods.

2.4 Enhanced Derived Features

```
python
# Price per square foot (key normalization metric)
df['price_per_sqft'] = df['price'] / df['sqft_living']

# Living-to-lot ratio (urban density proxy)
df['living_lot_ratio'] = df['sqft_living'] / df['sqft_lot']

# Age bucket (non-linear aging effects)
df['age_bucket'] = pd.cut(df['house_age'], bins=[0, 5, 15, 30, 60, 150],
                          labels=['new', 'recent', 'established', 'mature', 'historic'])

# Interaction features
df['grade_x_sqft'] = df['grade'] * df['sqft_living']
df['waterfront_x_view'] = df['waterfront'] * df['view']
```

2.5 Phase 2 Deliverables

- H3 hexagonal indexing at resolution 7 and 9
- Micro-market aggregate features (median price, price/sqft, sales volume per hex)
- Amenity proximity features (schools, transit, parks, downtown)

- Macroeconomic time-series features merged by sale month
- Final feature matrix saved as Parquet: `data/features/feature_matrix.parquet`
- Feature correlation analysis and visualization notebook

Phase 3 — Scalable ML Pipeline

> **TIP**
>
> LightGBM trains 3–5x faster than XGBoost on large datasets due to its histogram-based gradient computation and leaf-wise tree growth. Use it as the primary model and keep XGBoost for comparison.

3.1 Train/Validation/Test Split Strategy

```
python
# Temporal split (more realistic than random split for real estate)
# Train: sales before 2024-01-01
# Validation: sales Jan–Jun 2024
# Test: sales Jul–Dec 2024

train = df[df['sale_date'] < '2024-01-01']
val   = df[(df['sale_date'] >= '2024-01-01') & (df['sale_date'] < '2024-07-01')]
test  = df[df['sale_date'] >= '2024-07-01']
```

Why temporal split? Random splitting leaks future information into training. A temporal split simulates real-world deployment where the model only sees past data.

3.2 Model Training Matrix

Model	Purpose	Library	Key Hyperparameters
Linear Regression (Hedonic)	Interpretable baseline	`sklearn`	StandardScaler, all features
LightGBM Regressor	Primary high-accuracy model	`lightgbm`	`num_leaves=127, n_estimators=1000, learning_rate=0.03, min_child_samples=20`
XGBoost Regressor	Secondary comparison model	`xgboost`	`max_depth=8, n_estimators=800, learning_rate=0.03`
LightGBM Quantile (10th)	Risk floor prediction	`lightgbm`	`objective='quantile', alpha=0.1`
LightGBM Quantile (50th)	Median prediction	`lightgbm`	`objective='quantile', alpha=0.5`
LightGBM Quantile (90th)	Risk ceiling prediction	`lightgbm`	`objective='quantile', alpha=0.9`

3.3 Hyperparameter Tuning with Optuna

```
python
import optuna

def objective(trial):
    params = {
        'num_leaves': trial.suggest_int('num_leaves', 31, 255),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1, log=True),
        'n_estimators': trial.suggest_int('n_estimators', 300, 1500),
        'min_child_samples': trial.suggest_int('min_child_samples', 10, 100),
        'reg_alpha': trial.suggest_float('reg_alpha', 1e-8, 10.0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-8, 10.0, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
    }
    model = lgb.LGBMRegressor(**params, random_state=42, verbosity=-1)
    model.fit(X_train, y_train, eval_set=[(X_val, y_val)],
              callbacks=[lgb.early_stopping(50)])
    preds = model.predict(X_val)
    return np.sqrt(mean_squared_error(y_val, preds))

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=100)
```

3.4 Evaluation Metrics

Metric	Formula	What It Tells You
RMSE	$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$ n1∑(yi − y^i)2	Average prediction error magnitude (sensitive to outliers)
MAE	$\frac{1}{n}\sum\|y_i - \hat{y}_i\|$n1∑|yi − y^i|	Average absolute error (robust to outliers)
MAPE	$\frac{100}{n}\sum \left\| \frac{y_i - \hat{y}_i}{y_i} \right\|$ n100∑‖yiyi−y^i‖	Percentage error (business-interpretable)
R^2	$1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$1 − ∑(yi−y¯)2∑(yi−y^i)2	Proportion of variance explained
Quantile Coverage	% of actual prices within 10th–90th band	Risk band calibration

3.5 Phase 3 Deliverables

- Temporal train/val/test split implemented
- LightGBM + XGBoost point regression models trained
- Optuna hyperparameter tuning with ≥50 trials
- Three quantile regression models (10th, 50th, 90th percentile)
- Model comparison table (RMSE, MAE, MAPE, R^2 for all models)
- Best model serialized: `models/best_lgbm.txt`

Phase 4 — Explainability & Deep Analysis

> NOTE
>
> This phase transforms the project from "just a model" into a comprehensive analytical suite with interpretable explanations and investment-grade analysis.

4.1 TreeSHAP Feature Explanations

```
python
import shap

explainer = shap.TreeExplainer(best_model)
shap_values = explainer.shap_values(X_test)

# Global feature importance (bar plot)
shap.summary_plot(shap_values, X_test, plot_type="bar")

# Individual property explanation (waterfall)
shap.waterfall_plot(shap.Explanation(
    values=shap_values[0],
    base_values=explainer.expected_value,
    data=X_test.iloc[0],
    feature_names=X_test.columns.tolist()
))
```

Example Output for a Single Property:

```
Base Value (avg market):       $485,000
+ sqft_living (2,800 sqft):    +$62,000
+ grade (10/13):               +$45,000
+ h3_micro_median_price:       +$38,000
+ dist_to_school (0.3 km):    +$12,000
- house_age (45 years):        -$28,000
- condition (2/5):             -$15,000
──────────────────────────────────────────
= Predicted Price:             $599,000
```

4.2 Production-Grade Arbitrage Detector with Anomaly Filtering

```
python
# Step 1: Compute residuals
results['residual_pct'] = (results['predicted'] - results['actual']) / results['predicted']

# Step 2: Filter out anomalies (likely data errors or distressed sales)
from sklearn.ensemble import IsolationForest
iso_forest = IsolationForest(contamination=0.05, random_state=42)
results['is_anomaly'] = iso_forest.fit_predict(results[['actual', 'sqft', 'bedrooms',
'grade']])

# Step 3: Flag genuine hidden gems (underpriced AND not anomalous)
genuine_gems = results[
    (results['residual_pct'] >= results['residual_pct'].quantile(0.90)) &
    (results['is_anomaly'] == 1)  # 1 = normal, -1 = anomaly
].sort_values('residual_pct', ascending=False)
```

4.3 Automated Comparable Properties ("Comps") Engine

```
python
from sklearn.neighbors import NearestNeighbors

# Feature vector: [lat, lng, sqft_living, bedrooms, bathrooms, grade, house_age]
comp_features = ['lat', 'lng', 'sqft_living_scaled', 'bedrooms_scaled',
                 'bathrooms_scaled', 'grade_scaled', 'house_age_scaled']

nn = NearestNeighbors(n_neighbors=6, metric='euclidean')
nn.fit(df_sold[comp_features])

# For a target property, find 5 most similar recently sold homes
distances, indices = nn.kneighbors(target_property[comp_features].values.reshape(1, -1))
comps = df_sold.iloc[indices[0][1:]]  # exclude self
```

4.4 Spatial Price Trend Analysis

```
python
# Price appreciation heatmap by H3 hex over time
h3_trends = df.groupby(['h3_res7', 'sale_year']).agg(
    median_price=('price', 'median'),
    sales_count=('price', 'count')
).reset_index()

# Year-over-year appreciation rate per hexagon
h3_trends['yoy_appreciation'] = h3_trends.groupby('h3_res7')['median_price'].pct_change()
```

4.5 Phase 4 Deliverables

- Global SHAP feature importance bar chart
- Per-property SHAP waterfall explanations
- Isolation Forest anomaly filter integrated into arbitrage detector
- Automated comps engine (5 nearest comparable properties)
- Spatial price trend analysis by H3 hexagon over time
- Analysis notebook with all visualizations

Phase 5 — Interactive Dashboard & API

> IMPORTANT
>
> Both `streamlit` and `fastapi` are already installed in your environment. This phase delivers the user-facing application layer.

5.1 Streamlit Dashboard Architecture

```
```

5.2 Tab Descriptions

Tab 1 — 🗺️ Geospatial Explorer

- Interactive **PyDeck / Folium map** with H3 hexagons colored by median price.
- Click a hexagon → popup shows: median price, price/sqft, sales volume, YoY appreciation.

- Filter by state, city, price range, number of bedrooms.

Tab 2 — 💰 Property Valuator

- User inputs: address or lat/lng, bedrooms, bathrooms, sqft, grade, year built.
- Outputs:
    - **Point estimate** (LightGBM predicted price)
    - **Risk band** (10th–90th percentile range as a gauge chart)
    - **SHAP waterfall** explaining the valuation
    - **5 comparable properties** on a mini-map

Tab 3 — 💎 Hidden Gems Finder

- Table of top 50 underpriced properties (filtered for anomalies).
- Columns: address, listed price, predicted fair value, discount %, estimated ROI.
- Map view pinpointing gem locations with color-coded discount magnitude.

Tab 4 — 📊 Market Analytics

- Price trend line charts by state/city/zip over time.
- Sales volume bar charts.
- Mortgage rate overlay vs. median price.
- Top appreciating and depreciating neighborhoods (H3 hexagons).

5.3 FastAPI Valuation Endpoint

```
python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Real Estate Valuation API")

class PropertyInput(BaseModel):
    lat: float
    lng: float
    bedrooms: int
    bathrooms: float
    sqft_living: int
    grade: int
    year_built: int

class ValuationResponse(BaseModel):
    predicted_price: float
    floor_10pct: float
    median_50pct: float
    ceiling_90pct: float
    top_features: list[dict]

@app.post("/valuate", response_model=ValuationResponse)
def valuate(prop: PropertyInput):
    features = engineer_features(prop)  # H3, micro-market, proximity
    point_pred = point_model.predict(features)
    q10 = q10_model.predict(features)
    q50 = q50_model.predict(features)
    q90 = q90_model.predict(features)
    shap_vals = explainer.shap_values(features)
    return ValuationResponse(
        predicted_price=float(np.expm1(point_pred[0])),
        floor_10pct=float(np.expm1(q10[0])),
        median_50pct=float(np.expm1(q50[0])),
        ceiling_90pct=float(np.expm1(q90[0])),
        top_features=get_top_shap_features(shap_vals, features)
    )
```

5.4 Phase 5 Deliverables

- Streamlit app with 4 tabs (Geo Explorer, Valuator, Hidden Gems, Analytics)
- Interactive H3 hexagon map with price heatmap

- Property valuation form with risk band gauge and SHAP explanation
- FastAPI endpoint for programmatic valuation
- README with setup and run instructions

Required Installations

```
bash
pip install duckdb lightgbm h3 shap optuna fredapi osmnx pydeck folium
```

> **NOTE**
>
> `streamlit`, `fastapi`, `xgboost`, `pandas`, `numpy`, `matplotlib`, and `scikit-learn` are already available in your environment.

Final Project File Structure

```
BDA/
├── data/
│   ├── raw/                          # Original CSV downloads
│   ├── properties/                   # Partitioned Parquet files
│   │   ├── state=CA/sale_year=2023/
│   │   └── ...
│   ├── features/
│   │   └── feature_matrix.parquet    # Final engineered feature set
│   └── realestate.duckdb             # DuckDB analytical database
│
├── scripts/
│   ├── 01_ingest.py                  # Download & clean raw data
│   ├── 02_to_parquet.py              # Convert to partitioned Parquet
│   ├── 03_duckdb_setup.py            # Register Parquet views in DuckDB
│   ├── 04_feature_engineering.py     # H3, amenity, macro features
│   └── 05_train_models.py           # LightGBM + Quantile training
│
├── notebooks/
│   ├── 01_EDA.ipynb                  # Exploratory analysis on full dataset
│   ├── 02_Feature_Analysis.ipynb     # Correlation, importance, SHAP
│   ├── 03_Model_Comparison.ipynb     # All models side-by-side
│   └── 04_Arbitrage_Analysis.ipynb   # Hidden gems deep dive
│
├── models/
│   ├── best_lgbm.txt                # Serialized LightGBM model
│   ├── quantile_q10.txt
│   ├── quantile_q50.txt
│   └── quantile_q90.txt
│
├── app/
│   ├── streamlit_app.py             # Main Streamlit dashboard
│   └── api.py                        # FastAPI valuation endpoint
│
├── Advanced_Real_Estate_Valuation_Report.docx
├── Real_Estate_Valuation_Project.ipynb   # Original notebook (preserved)
└── README.md
```

Summary: What Each Phase Adds to Your BDA Grade

Phase	BDA Concepts Demonstrated
1	Big data storage formats (Parquet), columnar compression, partitioning, OLAP query engines (DuckDB), data ingestion pipelines
2	Spatial indexing (H3), feature engineering at scale, external data integration (APIs), geospatial analysis
3	Scalable gradient boosting (LightGBM), hyperparameter optimization (Optuna), quantile regression, temporal train/test splits
4	Explainable AI (SHAP), anomaly detection (Isolation Forest), automated comparable analysis (KNN), spatial trend analytics
5	Interactive dashboards (Streamlit), REST APIs (FastAPI), real-time inference, full-stack data science delivery